Software Development

Motivation + Administrative Information

Benjamin Hackl

Department of Mathematics and Scientific Computing @ University of Graz

About this course

⚠ Disclaimer

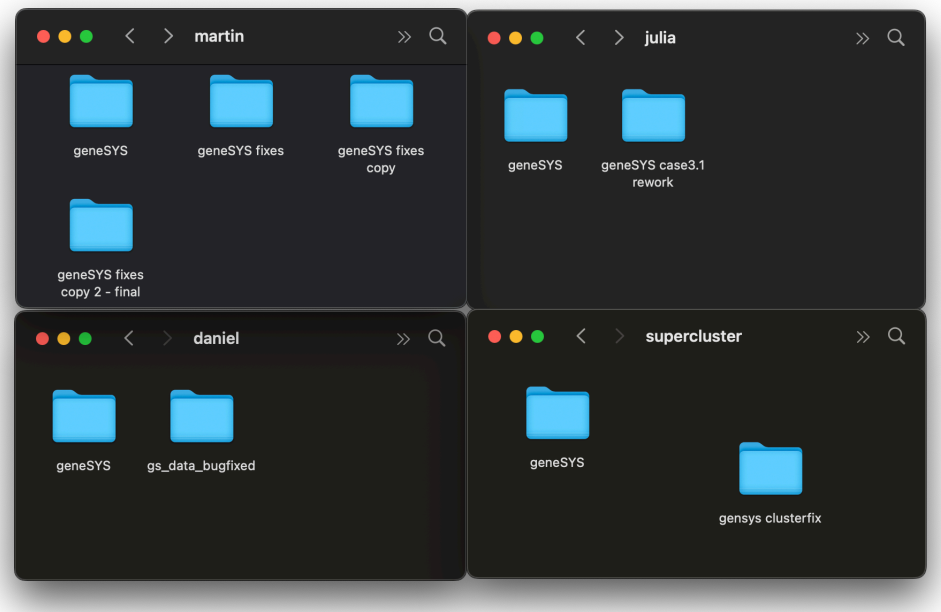
This course is **not** a coding 101 course: some proficiency is expected, we'll mostly focus on related *meta skills!*

- Coding / Programming vs. Software Development / Software Engineering?

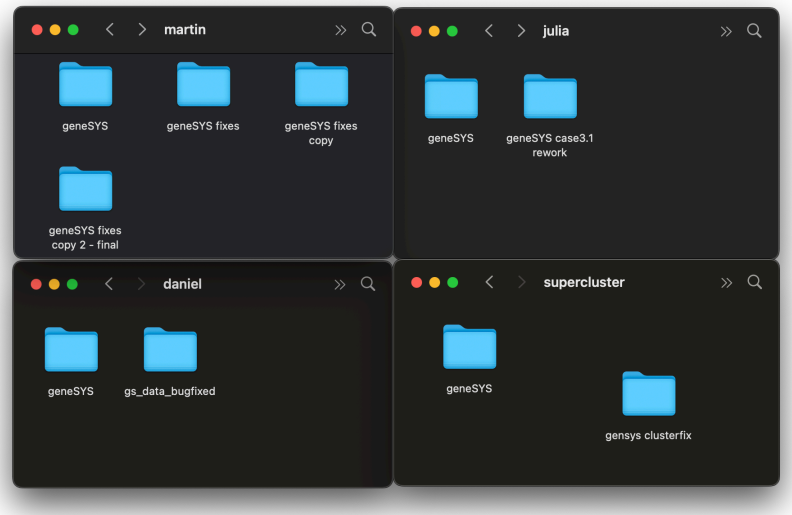


Example: `project-v5-feedbacked-final(2)` ...

- Daniel, Julia, and Martin are working on a joint project
- Martin writes some code for computations on a cluster



Example: `project-v5-feedbacked-final(2)` ...



They end up with:

- a mess (seriously)
- computation time spent running code missing some fixes
- no way to reproduce the exact code that they used to run a year later

Example: A Short Story about Netscape

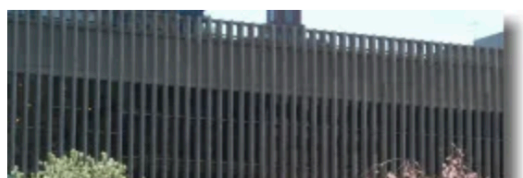
- *Netscape Navigator* dominated the browser market in the mid-1990s
- In late 1998, the planned 5.0 was abandoned in favor of a rewrite based on the new *Gecko* rendering engine…
- … no final 5.0 release ever shipped. ☹
- v6.0 arrived in November 2000 – over 3 years after v4.0; the old 4.x line still received updates during that time 💀



Netscape Navigator on Windows 3.1, mid-1990s / Wikimedia

Example: A Short Story about Netscape / 2

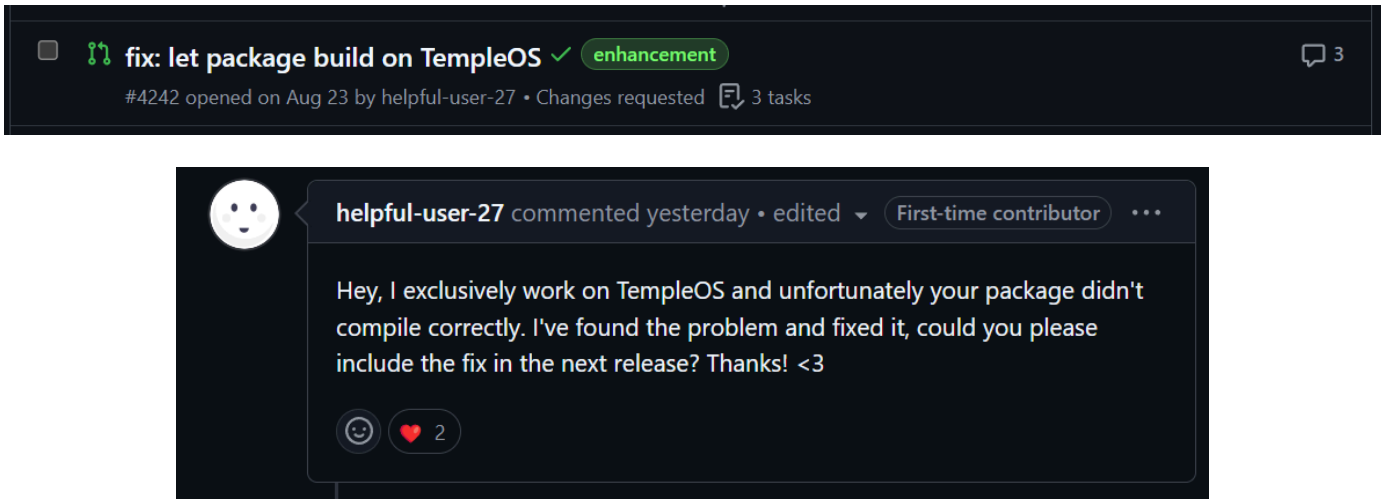
From Joel Spolsky's Blog "Things You Should Never Do":

> It's important to remember that when you start from scratch there is **absolutely no reason** to believe that you are going to do a better job than you did the first time. First of all, you probably don't even have the same programming team that worked on version one, so you don't actually have "more experience". You're just going to make most of the old mistakes again, and introduce some new problems that weren't in the original version.
>
> 
>
> The old mantra *build one to throw away* is dangerous when applied to large scale commercial applications.

Rewrites risk losing accumulated knowledge and bug fixes – this is a cautionary case study, not a rule that every rewrite is wrong.

Example: Can you please quickly merge this?

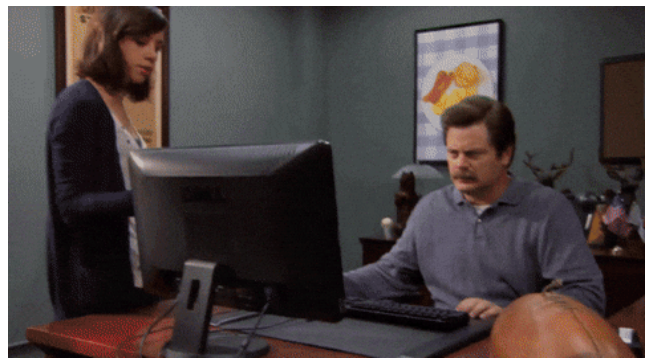
- Emma maintains a small package; most other developers have left already



- She does not have access to a TempleOS-machine & skips testing
- releases a new version with these changes included

Example: Can you please quickly merge this? / 2

- Next morning: Emma wakes up to 742 (angry) messages in her inbox



- Her package is broken everywhere (except on TempleOS)
- Downstream projects relying on her project are now broken too
- Technical fixup: yank release (if supported), revert change, publish a fix
- Project cleanup: answering emails, dealing with opened issues, … (rest of the week)

This course is about...

... learning and discussing tools that are helpful in the context of *(sustainable) software development*. This includes:

- navigating your computer / remote servers using a shell,
- version control and practical collaboration models,
- setting up software projects
- creating reproducible development and runtime environments / container technologies,
- documentation, documentation, documentation, and the FAIR principle,
- useful tools for data science,
- and more!

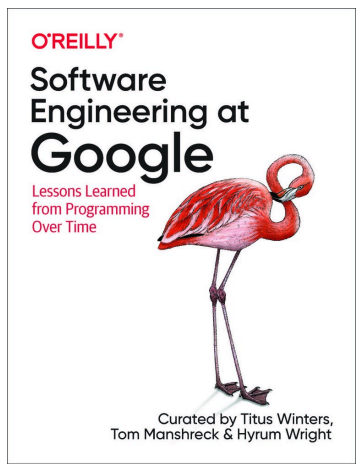
References and Resources

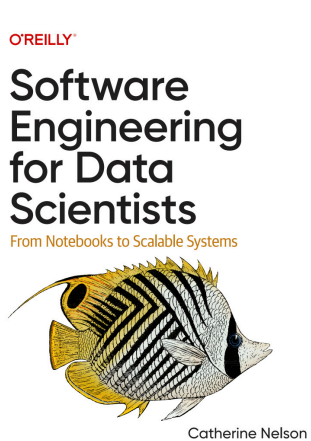
This course is based on material from various sources (and you will find much more out there):

- MIT's Missing Semester Course
- LV *Software Management for Scientific Computing* @ University of Innsbruck
- Various manuals and project documentation pages …

Recommended Reading

- *Software Engineering at Google*, by Titus Winters et al. – online available!
- *Software Engineering for Data Scientists*, by Catherine Nelson





Preliminary Schedule

Date	Topic
2026-10-05	Motivation, Administrative Information, Shell
2026-10-13	Shell (ctd.), Version Control and Git
2026-10-20	Python Packaging and Tools / I
2026-10-27	Agentic Coding Workshop
2026-11-03	Python Packaging and Tools / II
2026-11-10	Reproducible Environments, Testing, Deployment: CI/CD
2026-11-17	How to Collaborate on Software
2026-11-24	Licenses and FAIR Software Practices

Afterwards: **Project Phase!**

Pop Quiz!

https://ars.uni-graz.at/p/45974049

What do I need to do to get which grade?

Full information about assessment can be found in our Moodle course.

- Two types of point sources: *exercises* (30 points) and a *group project* (30 points).
- Additionally, up to 6 *bonus points* can be earned.
- At least **15 points need to be earned in each individual part** to obtain a passing grade.

Total points	Grade
≥ 30, < 37.5	Genügend (4)
≥ 37.5, < 45	Befriedigend (3)
≥ 45, < 52.5	Gut (2)
≥ 52.5	Sehr gut (1)

Exercise and bonus points

- There will be **7 exercise sheets** containing assignments and problems related to the content of the lecture part.
- Until **Friday (at 10:00)**, you can mark online which exercises you have prepared and uploaded to earn $5 \cdot \frac{\text{marked exercises}}{\text{exercises on sheet}}$ points.
  - That is, up to 5 points per exercise session,
  - only the 6 best sessions are considered for your overall score.
- Exercises are checked randomly, ticking problems that were not properly solved will lead to deduction of some (or all) points this week.
- Presentations can yield up to 2 bonus points each (6 points max).
- Attendance: only mandatory if you have ticked exercises.

Thoughts behind the weekly exercises

- This course touches various topics that you are often just expected to have picked up yourself at some point (e.g., working in a terminal, version control, …).
- Just like with programming, practical exercises are essential for learning these skills.
- Try and solve the problems on your own, in particular: don't just ask AI to solve the assignments for you.
  - Do experiment & use LLMs or agentic coding to learn – but make sure **that you understand what you are presenting.**
  - Don't be a *meat proxy* for your LLM.
- University-provided AI chat for students: studiGPT (log in with your Uni Graz account).

Project Assignment

- Around mid-semester you will be divided into small groups (of ~4-5 students each) to work on a software project assignment.
- Some project ideas will be provided, your own suggestions are welcome too.
- Projects should touch multiple lecture topics and ideally involve an (open) data source.
  - A draft for detailed project requirements will be made available soon in our Moodle course.
- Points can be earned for…
  - the project itself (5 points) and an end-of-semester demo (10 points),
  - and a corresponding *extended devlog* (15 points).

Project Assignment: Extended Devlog

The *extended devlog* includes

1. the VCS history including corresponding pull requests + code reviews (5 points)
2. a short written report summarizing how your group has approached the project from start to finish: conceptualization, writing specifications, coding, code reviews, release(s) (10 points)

> **Use of external tools**
>
> Make use of whatever tools you see fit. List your tools explicitly in the devlog. In particular, **document explicitly** wherever you use generative AI.

Thoughts behind the project assignment

- Very important: **do not** plan the project as a last-minute overnight assignment; coding part is not that important – trying out real-world collaboration techniques is!
- No exercise sessions in 2nd half of semester: replaced by (bi)weekly individual group meetings to discuss progress.
- Define clear responsibilities; document when things go wrong (the classic issue is underestimating required implementation time…)
- **Failing is okay!** The important part is to learn from your mistakes and to document your findings in the devlog.

Any Questions?